

单个微服务设计工作流

阶段一：场景库

产出：一份完整的场景列表，覆盖所有正常、异常、边界情况。

工作方式：逐条提出场景，检查有没有遗漏的隐含场景，直到穷举完毕。

完成标准：给任何一个新场景，都能在列表里找到它属于哪一类，没有空白地带。

阶段二：边界与约束

产出：系统不做什么（5-8条）+ 约束列表（从场景推导，每条注明来源场景）。

工作方式：从场景库自动推导初稿，逐条确认、修改或否定。

完成标准：约束之间没有矛盾，每条约束都能追溯到至少一个场景。

阶段三：执行可靠性模型

单独成节，先于所有包设计写完。内容就是我们今天讨论的那四件事，但此时有了场景和约束支撑，每一条都有出处。

完成标准：给任何一个崩溃场景，都能从模型里推导出 Runtime 的行为，不需要查代码。

阶段四：逻辑架构

产出：模块划分 + 每个模块的职责边界 + 模块间关系。

工作方式：从约束推导，不是从直觉设计。每个模块的边界都能用至少一条约束解释。

完成标准：任意一个场景，能在逻辑架构里走通完整路径。

阶段五：数据架构

产出：核心数据模型（在 core.model 基础上调整）+ 每个字段的存在理由。

工作方式：每个字段必须能追溯到一个场景或约束，没有来源的字段不允许存在。

完成标准：给任意一个场景，能在数据模型里找到承载它所需的所有字段。

阶段六：包设计

每个包单独一轮对话，顺序是 common → engine → strategy → api。

每个包的设计流程：

1. 列出这个包需要承接的场景

2. 从场景和约束推导职责
3. 定义接口和数据结构
4. 我用约束列表验证，你确认

完成标准：包内每个类的职责都能用一句话说清楚，且不和其他包重叠。

贯穿全程的规则

每个阶段结束前不进入下一阶段，这是防止失控的唯一机制。

每次对话只推进一个阶段内的小问题，你是个人开发者，慢和细致就是最快的方式，这条规则保护这个节奏。

所有决策记录原因，不只记结论。格式就是"因为场景X有约束Y，所以决定Z"。

现在可以回到阶段一，继续补充场景库了。今天已经列了9个，从你认为最重要的缺口开始。